

# Web Traffic Classification with Extremely Randomized Trees

CSCI434 COLL 400 Capstone Project

Ramsey Jooss

rrjooss@wm.edu

William & Mary

Williamsburg, Virginia, USA

## ABSTRACT

A clear and well-documented  $\LaTeX$  document is presented as an article formatted for publication by ACM in a conference proceedings or journal publication. Based on the “acmart” document class, this article presents and explains many of the common variations, as well as many of the formatting elements an author may use in the preparation of the documentation of their work. A cite so this compiles [?] ].

## KEYWORDS

Decision and regression trees, Ensemble methods, Extremely randomized trees, Machine Learning, Network Traffic Classification

## 1 INTRODUCTION

Traffic classification "is instrumental to a number of activities that are of extreme interest to carriers, Internet service providers and network administrators in general" [13]. Networking's ubiquity, and the role it plays therein means traffic classification is a vital consideration across contexts and domains. "Indeed, [it] is the basic block that is required to enable any traffic management operations, from differentiating traffic pricing and treatment (e.g., policing, shaping, etc.), to security operations (e.g., firewalling, filtering, anomaly detection, etc.)" [13]. The variation across contexts means possible approaches to the problem also vary, both across time and domain.

In cloud computing (specifically in the context of micro-service architecture) envisaging service meshes as a new layer in the network stack has been proposed. "Lower layers[visibility is limited] due to lacking context of API calls, encryption, and" multiplexing or API aggregation among other things. Limited visibility means less knowledge of applications' needs and therefore less responsiveness to them in this network-intensive context [1].

At the same time old methods like port-based classification and payload-based classification are becoming less possible with the proliferation of encryption and use of non-standard or random ports [13]. The former applies to web traffic, while an analogous issue exists for the latter in the prevalence and

diversity of web traffic. In response and particularly empowered by the diverse nature of web services, and correspondingly their traffic, statistical classification, done on the packet or behavioral level, can be leveraged [13].

This project collected a dataset of packet capture records of web traffic from: Google.com, Github.com, Yahoo.com and GeeksforGeeks.com. First this data was split into training and test sets in a 8:2 ratio. An extremely randomized forest model was used for classification [6] with two resampling methods for validation: Bootstrap sampling and repeated k-Fold Cross Validation [11].

The goal of this project is to apply machine learning techniques and tools from elsewhere alongside knowledge learned in CSCI434 to a real-world scenario.

## 2 METHOD

### 2.1 Data Preparation

This section presents the methodology for generating, analyzing and classifying web traffic based on its originating domain. This pipeline is depicted in Fig 1.

**2.1.1 Target Enumeration.** To generate a diverse targets for each of the target domains two methods were used sitemap parsing and spiderfoot [9]. The python package *Ultimate Sitemap Parser* [3] was used for generating lists of pages from Google.com, Yahoo.com and GeeksforGeeks.com. However, this method was not possible for Github.com since it lacks a sitemap. Instead, the open source intelligence (OSINT) automation tool Spiderfoot was used.

These techniques generated many thousands of locations for each domain. Using all of these would be unfeasible for the scope of this project to they were arbitrarily reduced to a random subset for each domain, totaling 495 urls.

**2.1.2 Traffic Generation.** Web traffic generation for systematic analysis poses an issue as manual web browsing is difficult at scale and risks the consistency of data, conversely naive programatic approaches of generating http(s) request

Figure 1: Data Preparation Pipeline



fail to emulate real web browsing. To programatically generate realistic web traffic this project uses Selenium WebDriver, "commonly referred to as just *WebDriver*" [2].

"WebDriver drives a browser natively, as a user would, either locally or on a remote machine using the Selenium server" [2]. This project uses WebDriver locally because of its limited scope, but leveraging the Selenium server may be useful for more extensive data collection. For the sake of simplicity this project opted for the simple emulation of loading the enumerated locations so only a subset of WebDriver's features were used. Still, its feature set provided an important tools for the task. One of "the most common challenge for browser automation is ensuring that the web application is in a state" before executing a particular command to avoid race conditions [2]. To address this WebDriver provides a diversity of waiting strategies [2], which this project uses to ensure the normal course of page loading is captured. The start and end epoch times for each target's web traffic was recorded so captured traffic's could be associated with its source.

**2.1.3 Traffic Capture.** To capture the generated network traffic *tcpdump* [8] was used. *Tcpdump* is based on *libpcap* [8], the same library *Wireshark* uses to capture live network data [7]. The choice between these utilities is arbitrary.

**2.1.4 Feature Engineering.** *Wireshark*'s cli *TShark* [5] was used to filter and extract timing and size details about each captured packet. Specifically the fields *frame.time\_epoch*, *frame.time\_delta*, and *tcp.len* were captured.

The *pandas* package [4] was used to first associate each packet data entry with its originating target and then to generate summary statistics of each target's set of entries. In addition to counting the number of packets for each target quartile range data, as well as mean and standard deviation for time deltas and *tcp* packet length were calculated. Summary statistics that were constant across all recorded entries were removed as they cannot be useful for categorization. Mode was not included for this reason since

Table 1: Resulting Records

| Domain            | Count |
|-------------------|-------|
| Google.com        | 68    |
| Github.com        | 71    |
| Yahoo.com         | 57    |
| GeeksforGeeks.com | 62    |

This step was lossy resulting in a limited data set of traffic from 258 targets, this could be the result of errors at either the Traffic Capture, or Feature Engineering stages but both were costly in terms of time so deeper analysis is not available at this time. The distribution of these records can be seen in Table 1.

## 2.2 Machine Learning for Classification

**2.2.1 The Extra-Trees Algorithm.** "Ensemble methods combine the predictions of several base estimators built with a given learning algorithm in order to improve generalizability / robustness over a single estimator" [12]. A traditional example of this is the random forests algorithm. "During the construction of each tree in the forest, a random subset of the features is considered" [12] and the most discriminative threshold is chosen for that node. The constructed trees are individually overfit which can lead to high variance, sensitivity to fluctuations in training data [12]. The final prediction is derivate by aggregating the outputs of an ensemble of unpruned trees via majority vote for classification or via averaging for regression [6]

Extra-Trees (Extremely Randomized Trees) is another ensemble learning method for regression or classification that extends the concept of randomized decision trees. Unlike traditional tree-based ensemble methods like Random Forests, Extra-Trees introduces a higher degree of randomization by selecting both the attribute and cutoff point entirely at random rather than optimizing split points locally towards maximizing a criterion [6] "This usually allows to reduce the

**Table 2: Arguments**

| ExtraTreesClassifier<br>Parameter | Default<br>Value | Parameter Grid<br>Options |
|-----------------------------------|------------------|---------------------------|
| n_estimators                      | 100              | 10, 20, 50, 100, 200      |
| max_depth                         | None             | None, 5, 10, 20, 30       |
| min_samples_leaf                  | 1                | 1, 5, 10                  |
| min_samples_split                 | 2                | 2, 5, 10                  |
| bootstrap                         | False            | True, False               |

variance of the model a bit more, at the expense of a slightly greater increase in bias" [12]. The scikit-learn python package's ExtraTreesClassifier class [12] was used to apply this algorithm.

**2.2.2 Validation Methods.** The two resampling methods used in this project are k-fold cross validation and bootstrap resampling. Empirical investigation suggests k-fold cross-validation exhibits lower variance than the bootstrap method [10]. Unfortunately this projects implementation does not lend itself to direct comparison because of the structural difference of implementation in the scikit-learn python package.

**The Bootstrap Method.** Bootstrap resampling "randomly generates training and validation sets" by randomly selecting rows of the training set with replacement [11]. This means that entries can be repeated in the final training set and the validation set consists of entries in the training set not contained in the final training set.

This validation method was employed by passing an argument to the ExtraTreesClassifier constructor.

**k-Fold Cross Validation.** "K-fold cross-validation assumes randomly dividing a dataset into k folds or parts of roughly equal size. The k-th fold is used as a validation set and the remaining k-1 folds are used for training the model. This procedure is repeated k times, and each time a different fold is used as validation set" [10]. "After iterating k times this way, an average of the k validation errors is calculated so that, again, a more accurate estimate of error rate can be obtained" [11]. This accuracy allows for comparing the results of varied parameters for the Extra-Trees algorithm to increase model accuracy.

This method was employed by using scikit-learn's GridSearchCV algorithm [12]. The parameter grid used, and the default values for those parameters are shown in Table 2.

### 3 EVALUATION

The summary of 50 runs with each validation method are shown in Table 3. Accuracy was tracked because it was used as the scoring parameter for GridSearchCV. These results

suggest that this implementation's k-Fold validation is insignificantly more accurate than the bootstrap method alone, but bootstrap resampling exhibited lower variance in accuracy.

Since the split between training and testing data is a randomized 3:1 split for each run, and both the bootstrap method and k-fold cross-validation heavily depend on the representativeness of training data, averaged data is more likely to give an accurate picture of the effectiveness of each approach than range data given the limited data set [10]. The similarity of the results between the two methods could suggest a limitation of the testing data. While the diversity of web traffic makes statistical methods of categorization all the more imperative, categorization at the domain level still leaves a large diversity of traffic in each category.

### 4 CONCLUSION & FUTURE WORK

Structurally, further developing the implementations for both the bootstrap and k-fold methods for better comparability is needed. Additionally other techniques for summarizing network behavior are needed. This could include cluster analysis, regression to match more complex patterns, and particularly in the interest of real world applicability statistics dependent on order and overall timing. Developing this more complex analysis would pair well with more complex interactivity in generating web traffic or finding real-world sources of data as techniques would likely have to be tuned to, and selected for, particular data patterns.

Further analysis of data collection methods is needed to improve it so a more diverse dataset can be efficiently compiled as to support even further analysis. Varying scales and diversity of data is only one axis along which further comparison of validation methods is needed. More measures of model fit also need to be considered such as precision, recall, and measures considering resultant forest / tree structures and prediction confidence.

### REFERENCES

- [1] Sachin Ashok, P. Brighten Godfrey, and Radhika Mittal. 2021. Leveraging Service Meshes as a New Network Layer. In *Proceedings of the Twentieth ACM Workshop on Hot Topics in Networks (HotNets '21)*. ACM, 229–236. <https://doi.org/10.1145/3484266.3487379>
- [2] Software Freedom Conservancy. 2024. Selenium | WebDriver. <https://www.selenium.dev/documentation/webdriver/>.
- [3] Ultimate Sitemap Parser Contributors. 2025. Ultimate Sitemap Parser. <https://ultimate-sitemap-parser.readthedocs.io/en/latest/>.
- [4] NumFOCUS Foundation. 2026. pandas. <https://pandas.pydata.org/>.
- [5] Wireshark Foundation. 2026. Wireshark. <https://www.wireshark.org/>.
- [6] Pierre Geurts, Damien Ernst, and Louis Wehenkel. 2006. Extremely randomized trees. *Machine Learning* 63, 1 (March 2006), 3–42. <https://doi.org/10.1007/s10994-006-6226-1>
- [7] The Tcpdump Group. 2026. Packet capture library (libpcap). <https://wiki.wireshark.org/libpcap>.
- [8] The Tcpdump Group. 2026. TCPDUMP LIBCAP. <https://www.tcpdump.org/>.

**Table 3: Summary of Results of 50 Runs**

| Method    | Avg Acc. | Acc. $\sigma^2$ | Most Common Final Parameters                                                                   |
|-----------|----------|-----------------|------------------------------------------------------------------------------------------------|
| k-Fold    | 78.0%    | 0.00378         | bootstrap: False, max_depth: None, min_samples_leaf: 2, min_samples_split: 5, n_estimators: 50 |
| Bootstrap | 77.5%    | 0.00229         |                                                                                                |

- [9] Offsec Services Limited. 2025. Spiderfoot. <https://www.kali.org/tools/spiderfoot/>.
- [10] Milica Ljumovic and Michael Klar. 2015. Estimating expected error rates of random forest classifiers: A comparison of cross-validation and bootstrap. In *2015 4th Mediterranean Conference on Embedded Computing (MECO)*. IEEE, 212–215. <https://doi.org/10.1109/meco.2015.7181905>
- [11] Robbie T. Nakatsu. 2021. An Evaluation of Four Resampling Methods Used in Machine Learning Classification. *IEEE Intelligent Systems* 36, 3 (May 2021), 51–57. <https://doi.org/10.1109/mis.2020.2978066>
- [12] scikit-learn developers. 2026. 1.11. Ensembles: Gradient boosting, random forests, bagging, voting, stacking — scikit-learn.org. <https://scikit-learn.org/stable/modules/ensemble.html#random-forests-and-other-randomized-tree-ensembles>.
- [13] Silvio Valenti, Dario Rossi, Alberto Dainotti, Antonio Pescapè, Alessandro Finamore, and Marco Mellia. 2013. *Reviewing Traffic Classification*. Springer Berlin Heidelberg, Berlin, Heidelberg, 123–147. [https://doi.org/10.1007/978-3-642-36784-7\\_6](https://doi.org/10.1007/978-3-642-36784-7_6)